Report

Shortest Route from Water Depot to a Repair Location using Dijkstra's Algorithm

1. Problem Statement

Develop a C++ program to represent a water depot, road junctions, and repair locations as vertices with non-negative travel distances as edge weights. Apply Dijkstra's algorithm to determine the least-cost route to a selected repair location and display the route and total distance. Modify the program to select a different source vertex and recalculate the shortest distances.

2. Objective

To represent a water supply road network as a weighted graph, with the water depot, road junctions and repair locations as vertices and roads as edges, and apply Dijkstra's algorithm to find the least-cost route and total distance from a selected source to a repair location, recalculate the shortest distances for a different source vertex, handling invalid locations and unreachable routes.

3. Algorithm

Step 1: START.

Step 2: Build the graph as an adjacency list: each water depot, road junction and repair location is a vertex and each road is an undirected edge (u, v) with a non-negative weight w (distance in km).

Step 3: Read the source vertex (the Water Depot, or any other location chosen by the user) and the repair location. If either location is not present in the graph, or the destination is not a repair location, display an error and STOP.

Step 4: For every location, initialize dist[] = INFINITY, parent[] = -1 and visited[] = false.

Step 5: Set dist[source] = 0 and push (0, source) into a min-priority queue.

Step 6: WHILE the priority queue is not empty, repeat Steps 7 to 9. When it becomes empty, go to Step 10.

Step 7: Remove the pair (d, u) with the smallest distance from the queue. If visited[u] = true, skip it and go back to Step 6.

Step 8: Mark visited[u] = true.

Step 9: For each unvisited neighbour v of u with edge weight w: if dist[u] + w < dist[v], then set dist[v] = dist[u] + w and parent[v] = u, and push (dist[v], v) into the queue.

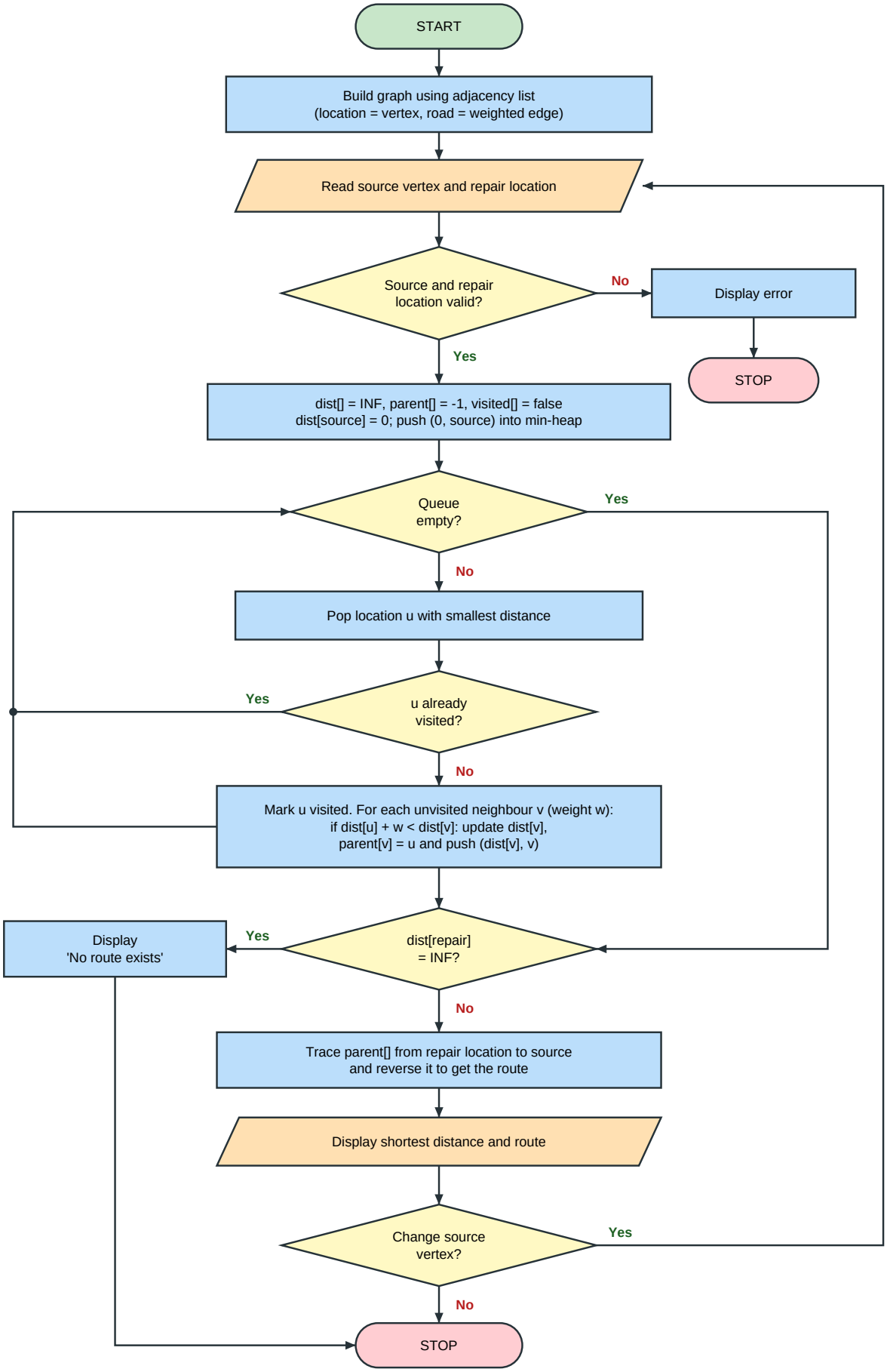
Step 10: If dist[repair location] = INFINITY, display "No route exists" and go to Step 13.

Step 11: Build the route: starting from the repair location, follow parent[] until the source is reached (parent = -1), then reverse the sequence.

Step 12: Display dist[repair location] as the shortest distance and the reversed sequence as the location route.

Step 13: To select a different source vertex, read the new source and repeat Steps 4 to 12, displaying the recalculated shortest distances. Otherwise, STOP.

4. Flowchart


START
Build graph using adjacency list
(location = vertex, road = weighted edge)
Read source vertex and repair location
Source and repair
location valid?
No
Display error
STOP
Yes
dist[] = INF, parent[] = -1, visited[] = false
dist[source] = 0; push (0, source) into min-heap
Queue
empty?
Yes
No
Pop location u with smallest distance
u already
visited?
Yes
No
Mark u visited. For each unvisited neighbour v (weight w):
if dist[u] + w < dist[v]: update dist[v],
parent[v] = u and push (dist[v], v)
Display
'No route exists'
Yes
dist[repair]
= INF?
No
Trace parent[] from repair location to source
and reverse it to get the route
Display shortest distance and route
Change source
vertex?
Yes
No
STOP